

HAND GESTURE CONTROLLED ROVER

Component Selection · Hardware Design · System Integration

DOCUMENT TYPE

Technical
Hardware Reference

CLASSIFICATION

Open
Academic Use

PURPOSE

Personal Project
Documentation

ASSEMBLED & BUILT BY

Sanjay Kalagarla

B.Tech · Computer Science & Engineering · IIT Patna

TABLE OF CONTENTS

1.	Project Overview	3
2.	System Architecture	3
3.	Bill of Materials & Component Functions	4
4.	Subsystem Breakdown	5
5.	Wiring & Integration Notes	7
6.	Assembly Guidelines	7
7.	Design Considerations & Constraints	8
8.	Conclusion	8
9.	Software Environment & Dependencies	9

1. Project Overview

The Hand Gesture Controlled Rover is a wireless, real-time robotic platform that translates hand gestures of an operator into directional commands for a ground vehicle. The system eliminates conventional push-button or joystick controllers by leveraging an IMU-equipped transmitter unit worn on the operator's hand, communicating via a 2.4 GHz RF link with the rover's onboard controller. The rover is capable of omnidirectional motion through differential drive, powered by a Li-ion battery pack and regulated DC-DC conversion. This document provides a complete hardware reference for replication, prototyping, and academic evaluation of the system.

Communication	2.4 GHz NRF24L01 RF Transceiver
Drive System	Differential Drive – 4 × DC Gear Motors (12V, 850 RPM)
Power Supply	Li-ion Battery Pack (4-cell, 2-cell configs) with DC-DC Buck Regulation
Controller	ESP32 (DevkitC-32UE) – Dual-core, Wi-Fi + BT capable MCU
Motor Driver	L298N Dual H-Bridge Motor Driver Module
PCB	1 × General-purpose PCB (25 cm × 5 cm)

2. System Architecture

The system comprises two independent nodes: a **Transmitter (Glove Unit)** worn by the operator and a **Receiver (Rover Unit)** mounted on the chassis. The transmitter captures hand orientation data via the MPU6050 IMU and encodes directional intent, which is wirelessly transmitted through the NRF24L01 module. The rover's ESP32

decodes the received packets and drives the L298N motor controller accordingly. Both nodes are powered by independent Li-ion cells with buck-converted regulated rails.

TRANSMITTER NODE		RF LINK		ROVER NODE	
MPU6050 IMU ↓ ESP32 (DevkitC) ↓ NRF24L01 TX	■■■■■	2.4 GHz	■■■■■	NRF24L01 RX ↓ ESP32 (DevkitC) ↓ L298N → Motors	■■■■■

3. Bill of Materials & Component Functions

The following table enumerates all hardware components used in the rover system. Quantities reflect a complete build of one rover unit plus one transmitter glove unit.

S.No	Component	Specification / Qty	Role in Rover
1	Mounting Screw Set	Assorted M3 – 1 set	Chassis assembly: securing PCB, motor mounts, and cover panels to the chassis frame.
2	Chassis – Fit-1 Frame	1 unit	Structural skeleton of the rover; houses all electronic modules and supports the drive train.
3	DC Gear Motor	12V, 3 kg·cm, 850 RPM – 4×	Primary actuators for wheel drive; 4-motor layout enables differential steering and turning.
4	Li-ion Cell Pack	4-cell + 2-cell configs	Main energy source for the rover (4-cell) and transmitter unit (2-cell); provides sustained runtime.
5	MPU6050 IMU	6-DOF – 1 unit	Mounted on transmitter glove; captures roll/pitch angles to derive gesture-based movement commands.
6	Li-ion Cell Holder	4-cell + 2-cell – 1 each	Holds battery cells in a fixed, safe arrangement; facilitates easy cell replacement.
7	DC-DC Buck Converter	Step-down module – 2×	Regulates battery voltage to stable 5V/3.3V rails for the MCU, sensors, and RF modules.
8	Heat Shrink Tubing	1mm, 2mm, 3mm – 1 m each	Insulates and protects all solder joints and bare wire connections across the rover and transmitter.
9	Silicone Wire (Multi-colour)	22–24 AWG – 1 m	High-flexibility interconnect wire for power distribution and inter-module signal routing.

S.No	Component	Specification / Qty	Role in Rover
10	Wheels	5 cm diameter – 4×	Coupled directly to motor shafts; 5 cm diameter chosen to balance speed and torque on flat terrain.
11	NRF24L01 Transceiver	2.4 GHz – 2×	Provides the bi-node wireless link; one unit on the transmitter, one on the rover receiver board.
12	NRF24L01 Adapter Module	Voltage adapter – 2×	Steps down and decouples the 3.3V supply to the NRF24L01, preventing MCU brownout during TX bursts.
13	Electrolytic Capacitor	47μF, 25V – 2×	Decoupling capacitors on motor driver power rails; suppress transient spikes during motor switching.
14	L298N Motor Driver Module	Dual H-Bridge – 1×	Drives all four DC motors under ESP32 PWM control; handles bidirectional current switching.
15	XT40 Connector Pair	Male + Female – 1 pair	High-current main battery connector; rated for peak motor inrush current during rover start-up.
16	ESP32-DevkitC-32UE	4 MB Flash – 2×	Central MCU on both transmitter and rover nodes; handles RF comms, gesture decoding, and PWM output.
17	B700 Hot-melt Adhesive	Glue sticks – 1 pack	Mechanical bonding of non-structural components; secures wire looms and lightweight sensor mounts.
18	General-purpose PCB	25 cm × 5 cm – 1×	Centralised prototyping board for soldering ESP32 breakout, connectors, and signal conditioning.
19	Female/Male Header Pins	2.54 mm pitch – 3 sets each	Socketed interconnects for ESP32, NRF24L01 adapter, and motor driver; enable modular swapping.

4. Subsystem Breakdown

4.1 Power Subsystem

The power architecture uses two independent Li-ion battery packs — a 4-cell configuration for the rover and a 2-cell configuration for the transmitter. Both packs feed dedicated DC-DC buck converters that supply regulated 5V to the ESP32 boards and 3.3V to the NRF24L01 modules via their adapter boards. 47μF decoupling capacitors are placed across the L298N motor driver supply to absorb the high-frequency transients generated during PWM-driven motor commutation. The XT40 connector pair provides a secure, high-current disconnect point between the battery and the power distribution bus.

4.2 Locomotion Subsystem

Four 12V, 850 RPM DC gear motors are arranged in a differential drive configuration — two motors per lateral side — all driven by a single L298N dual H-bridge module. The ESP32 generates PWM signals on the enable pins and sets direction via the IN1–IN4 digital lines. Forward, reverse, and point turns are achieved by setting appropriate motor polarities. The 5 cm wheels are press-fit or screw-coupled to the motor output shafts, yielding a practical linear speed suited for indoor and semi-outdoor navigation.

4.3 Wireless Communication Subsystem

Two NRF24L01+ modules operating in the 2.4 GHz ISM band form the RF link between transmitter and rover. The NRF24L01 adapter boards handle level-shifting from ESP32's 3.3V IO and provide local capacitive filtering to prevent supply-rail droop during transmission. The modules communicate via SPI at up to 2 Mbps on a defined pipe address, with auto-acknowledgement enabled to detect lost packets. Latency is kept sub-10 ms under line-of-sight conditions up to 10–15 m.

4.4 Gesture Sensing Subsystem (Transmitter)

The MPU6050 6-axis IMU is mounted on the dorsal side of the operator's glove. It communicates with the transmitter ESP32 over I2C and reports raw accelerometer and gyroscope data at 100 Hz. The ESP32 firmware applies a complementary filter to derive stable roll and pitch angles. Thresholded angle bands map to discrete command tokens (FORWARD, BACKWARD, LEFT, RIGHT, STOP) which are serialised and transmitted over the RF link on each update cycle.

4.5 Electronics Integration (PCB & Wiring)

All primary electronics — ESP32, motor driver interface headers, DC-DC output rails, and RF module socket — are mounted on a single 25 cm × 5 cm general-purpose PCB. Header sockets are used throughout to allow module replacement without desoldering. Silicone-insulated wire (22–24 AWG) handles all inter-board connections, chosen for its flexibility and resistance to chassis vibration fatigue. Heat-shrink tubing is applied to every solder joint to prevent shorts and improve mechanical durability.

5. Wiring & Integration Notes

Connection	Interface	Notes
ESP32 → NRF24L01 Adapter	SPI (MOSI, MISO, SCK, CSN, CE)	Use adapter board to avoid 3.3V noise; add 100μF cap on VCC.

Connection	Interface	Notes
ESP32 → MPU6050	I2C (SDA, SCL)	4.7kΩ pull-ups on SDA/SCL; AD0 pin → GND (addr 0x68).
ESP32 → L298N	4× GPIO (IN1–IN4) + 2× PWM (ENA, ENB)	PWM freq: 1–5 kHz; ensure common GND between ESP32 and L298N.
L298N → Motors	OUT1–OUT4 direct to motor terminals	Twist motor pairs; 47μF cap across motor terminals reduces EMI.
Battery → Buck Converter	XT40 → screw terminals	Set output to 5V before connecting ESP32; use multimeter to verify.

6. Assembly Guidelines

Step 1 – Chassis Preparation

Secure the four DC gear motors into the Fit-1 chassis frame using the provided mounting hardware. Ensure motor shafts protrude correctly before tightening fasteners.

Step 2 – Wheel Attachment

Press-fit or bolt the 5 cm wheels onto each motor shaft. Confirm free rotation without wobble.

Step 3 – PCB Preparation

Solder header sockets for the ESP32, NRF24L01 adapter, and L298N signal leads onto the general-purpose PCB. Mount the PCB in the chassis using standoffs and M3 screws.

Step 4 – Power Rail Setup

Connect battery packs to buck converters via the XT40 connector. Set converter outputs to 5V. Apply 47μF decoupling caps on L298N power input. Route power to ESP32 VIN pins.

Step 5 – Motor Driver Connection

Wire L298N OUT1–OUT4 to respective motor terminals. Connect IN1–IN4 and ENA/ENB to designated ESP32 GPIO pins.

Step 6 – RF Module Mounting

Plug NRF24L01 modules into adapter boards. Mount adapter in header socket on PCB. Ensure antenna has unobstructed line of sight.

Step 7 – Transmitter Assembly

Mount MPU6050 on glove. Connect to transmitter ESP32 via I2C. Plug in second NRF24L01 adapter. Power via 2-cell Li-ion + buck converter.

Step 8 – Firmware Flash & Test

Flash receiver firmware to rover ESP32 and transmitter firmware to glove ESP32 via USB. Verify RF pairing, then test gesture-to-motion mapping.

7. Design Considerations & Constraints

- **Power Budget:** Four 850 RPM motors at full load can draw 2–3A per motor. Size the battery capacity accordingly (minimum 2000 mAh for ~20 min runtime). The 2-cell transmitter pack is adequate given the low power draw of the ESP32 and NRF24L01.
- **RF Interference:** The NRF24L01 operates in the congested 2.4 GHz band. Select an RF channel (0–125) away from local Wi-Fi bands (channels 1, 6, 11 ≈ 2.412, 2.437, 2.462 GHz) to minimise packet loss.
- **Motor EMI:** DC brush motors generate significant electromagnetic noise. Fit 100nF ceramic capacitors across each motor terminal pair and ensure chassis ground is tied to the circuit common to suppress radiated EMI.
- **Firmware Latency:** Gesture command cycle time must remain below 50 ms for responsive control. Avoid blocking delays in the main loop; use hardware interrupts or DMA for I2C and SPI transactions.
- **Mechanical Alignment:** Misaligned motors introduce yaw drift even during forward commands. Calibrate PWM duty cycles per motor to achieve straight-line travel before gesture calibration.
- **Scalability:** The ESP32's dual-core architecture and available GPIO headroom permit straightforward addition of ultrasonic obstacle avoidance, a camera module (OV2640), or a PID speed controller without requiring a board change.

8. Conclusion

This documentation provides a complete hardware reference for the Hand Gesture Controlled Rover, covering the bill of materials, subsystem design rationale, wiring integration, and assembly procedure. The platform demonstrates a practical implementation of embedded wireless communication and inertial sensing for intuitive human-robot interaction. The modular architecture — centred on the ESP32 MCU and NRF24L01 RF link — allows the design to be extended into more complex robotic applications with minimal hardware revision. All components selected are commercially available, cost-effective, and widely supported in the open-source embedded community, making this an accessible build for academic teams and independent developers.

Note: This document is intended for academic and educational purposes. Builders should follow standard electronics safety practices — particularly when working with Li-ion batteries and high-current motor circuits. Always verify voltage levels before connecting sensitive modules.

9. Software Environment & Dependencies

Both the transmitter (glove) and receiver (rover) nodes are programmed using the **Arduino IDE 2.3.10**. The following board support packages and libraries are required to compile and flash the firmware for this project. All packages are available through the Arduino Library Manager and Boards Manager.

9.1 Board Support Package

The ESP32-DevkitC-32UE is not supported by the default Arduino IDE installation. The following additional Board Manager URL must be added under **File** → **Preferences** → **Additional boards manager URLs** before installing the package:

Additional Board Manager URLs:
https://espressif.github.io/arduino-esp32/package_esp32_index.json

Board Package	Version	Installed Via	Purpose in Project
Arduino ESP32 Boards	2.0.18-arduino.5	Boards Manager (Espressif URL above)	Provides full ESP32 core support including GPIO, SPI, I2C, PWM, and Wi-Fi/BT drivers for the DevkitC-32UE on both rover and transmitter nodes.

9.2 Arduino Libraries

The following libraries are installed via **Tools** → **Manage Libraries** in the Arduino IDE. Only the libraries directly used by the rover or transmitter firmware are listed below — other installed packages visible in the Library Manager are not required for this build.

Library	Version	Node	Role in Project
RF24 by TMRh20	1.6.1	Both nodes	Core driver for the NRF24L01 transceiver. Handles SPI communication, pipe addressing, auto-ACK, and packet transmission/reception on both the transmitter glove and rover receiver.
MPU6050 by Electronic Cats	1.4.5	Transmitter only	Provides I2C communication with the MPU6050 6-axis IMU. Used to read raw accelerometer and gyroscope data from which hand orientation (roll/pitch) is derived for gesture detection.
L298N by Andrea Lombardo	2.0.3	Rover only	Abstracts L298N H-bridge control — wraps PWM speed and direction commands for each motor channel, simplifying forward/reverse/turn logic in the rover's main control loop.
DCMotorController by Mauricio Caballero	0.1.2	Rover only (optional)	Alternate DC motor PWM helper. Used as a fallback if direct L298N library binding is not configured; provides speed ramping utilities for smoother motor acceleration.

9.3 Libraries Used During Initial Testing Phase

During the early development phase, the rover's motor and motion logic was first validated using an Arduino board paired with a Bluetooth module and a smartphone controller app, prior to migrating to the final ESP32 + NRF24L01 gesture-based architecture. The libraries below were installed and used during that testing stage. They are **not part of the final firmware** but are retained in the environment as a reference to the development process.

Library	Testing Context
---------	-----------------

HC0x_AT_Config v2.0.1	Used to configure the HC-05/HC-06 Bluetooth module via AT commands during the Arduino + Bluetooth phone-controller testing phase.
BTLE v1.0	Explored for Bluetooth LE communication between the Arduino and a smartphone controller app as an alternative control interface during early testing.
RF24Network v2.1.1	Installed alongside RF24 to explore multi-layer networking during RF link testing; ultimately not needed for the simple two-node setup.
RF24Mesh v2.1.1	Tested as part of evaluating RF24 stack capabilities during the wireless communication prototyping stage.
RF24Ethernet v2.1.4	Installed as a dependency during RF24 stack exploration; not actively used in any test sketch for this project.
nrf_to_nrf v1.4.7	Evaluated during RF module testing to understand cross-module communication options before finalising the NRF24L01 point-to-point approach.
Soldered nRF24 v1.0.1	Tested as an alternate NRF24L01 driver during early RF bring-up; the standard RF24 library was chosen for the final build.
lwIP v1.0.0	Installed during exploration of TCP/IP-over-RF networking as a potential high-level communication layer; not used in the final implementation.
RF24G v1.0.1	Evaluated for multi-radio scenarios during the prototyping phase; the single-pair RF24 setup proved sufficient for this application.

IDE Setup Note: After adding the ESP32 board manager URL and installing the board package, select **ESP32 Dev Module** (or ESP32-WROOM-DA Module) under **Tools** → **Board** → **ESP32 Arduino**. Set Upload Speed to **115200**, Flash Size to **4MB**, and the correct COM port before flashing either node.